## title: HoloCubic「AI 状态灵动岛」零基础使用指南（macOS + Wi-Fi）

# HoloCubic「AI 状态灵动岛」零基础使用指南

> 适用：HoloCubic 透明小电视（实测 ESP32-PICO-D4）+ HoloCubic_AIO 固件 v2.1.13 + 新增的 `AIStatus` APP（v0.2，USB 串口版）；电脑为 macOS。
>
> 效果：Claude Code / Codex / Grok 工作时（通过 **USB 数据线**，不需要 Wi-Fi），小电视实时显示 **谁、在干什么、多久了**，RGB 灯同步变色。
>
> 标注说明：**【已核对代码】** 表示根据仓库源码确认；**【未实机验证】** 表示还没有在真实硬件/Mac 上跑过，请以实际为准。

---

## 0. 整体原理（一分钟看懂）

```
AI 工具（Claude/Codex/Grok）──hook 触发──▶ ~/.holocube/holocube_notify.sh
      ──USB 串口 115200，一行 JSON──▶ 小电视 ──▶ 屏幕 + RGB 灯（回复 AIS:OK）
```

- 小电视用 USB 线连在 Mac 上即可，**不需要 Wi-Fi**。适合公司网络需要网页认证登录、设备连不上的情况。
- 状态和日志走同一个串口。小电视收到状态时，无论当前在菜单还是其他 APP，都会**自动切换进 AIStatus**（Settings APP 正在用串口时除外）。
- 同一时间串口只能被一个程序打开：开着串口监视器时，状态发不过去。

---

## 1. 准备

### 1.1 硬件

| 物品 | 说明 |
|---|---|
| HoloCubic 小电视 | 已组装好的 ESP32 版（Naive / Metal 均可）。 |
| USB 数据线 | **必须能传数据**。很多随手拿的线只能充电，插上后电脑完全没反应。 |
| microSD 卡（可选） | AIStatus **不需要** SD 卡，中文字体已内置（见 5.2）。相册、视频、天气图标等其他 APP 才需要。官方 Q&A 推荐爱国者 32G、闪迪 32G 普通卡。 |
| 2.4 GHz Wi-Fi（可选） | USB 模式**不需要** Wi-Fi；只有 Wi-Fi 模式（8.6）才需要，且不支持 5 GHz 和网页认证。 |
| 供电 | 电脑 USB 口或 5V/1A 以上充电头。供电不足时，连 Wi-Fi 那一刻容易重启。 |

### 1.2 Mac 软件

1. **VS Code**：https://code.visualstudio.com/ 下载后拖进「应用程序」。
2. **PlatformIO IDE 插件**：VS Code 左侧「扩展」(四个方块图标) → 搜 `PlatformIO IDE` → Install。装好后会自动下载工具链，第一次要几分钟到十几分钟，完成后重启 VS Code。
3. **Git**（可选）：终端输入 `git --version`。如果弹出安装「命令行开发者工具」的提示，点安装即可。

### 1.3 判断串口芯片、要不要装驱动

先**不插**小电视，打开「终端」(启动台搜 Terminal)，运行：

```
ls /dev/cu.*
```

再**插上**小电视，再运行一次，**多出来的那一项**就是小电视：

| 多出来的名字（示例） | 芯片 | 驱动 |
| --- | --- | --- |
| `/dev/cu.SLAB_USBtoUART` 或 `/dev/cu.usbserial-0001` | CP210x（CP2102） | 新版 macOS 通常免驱；不出现就装 Silicon Labs「CP210x VCP Mac」驱动 |
| `/dev/cu.wchusbserial*` 或 `/dev/cu.usbserial-14xx` | CH340 / CH343 | 新版 macOS 通常免驱；不出现就装 WCH 官方 macOS 驱动 |
| 什么都没多 | — | 先换**数据线**、换 USB 口；再装驱动；装完到「系统设置 → 隐私与安全性」点「允许」，然后重启 |

> 官方 Q&A 提到原版主板用 **CP2102**【已核对文档】。你的板子具体用哪颗芯片，以上表判断为准。
>
> 也可以在「关于本机 → 系统报告 → USB」里看到 `CP210x` 或 `USB Serial (CH340)` 字样。

---

## 2. 获取代码并打补丁

二选一即可。

### 方式 A：直接用打包好的 zip（最简单）

1. 拿到 `holocubic_ai_status.zip`，双击解压。
2. 里面有 `AIO_Firmware_PIO/`（固件工程，已包含 AIStatus）和 `ai_status_bridge/`（电脑端脚本）。

### 方式 B：从官方仓库 clone，再打补丁

```
cd ~
git clone https://github.com/ClimbSnail/HoloCubic_AIO.git
cd HoloCubic_AIO
```

```
git apply --check ~/Downloads/ai_status.patch   # 先检查，不报错再继续
git apply ~/Downloads/ai_status.patch
```

补丁的改动如下：

- 新增 `AIO_Firmware_PIO/src/app/ai_status/`（5 个文件）。
- 修改 `src/app/app_conf.h`：加 `APP_AI_STATUS_USE 1`。
- 修改 `src/HoloCubic_AIO.cpp`：把 AIStatus 安装为菜单第一个 APP。
- 修正 LHLXW APP 中 3 处头文件大小写（`arduino.h`→`Arduino.h`、`EMOJI.h`→`emoji.h`）。
- 新增 `ai_status_bridge/`（电脑端脚本与说明）。

补丁基于 2026-10-14 的仓库主分支。官方之后有更新时，可能需要手动合并。

---

## 3. 刷固件

### 3.1 打开工程

VS Code → `File → Open Folder…` → 选择 **`AIO_Firmware_PIO`** 这一层文件夹（里面直接能看到 `platformio.ini`）。选错层级 PlatformIO 不会识别。

### 3.2 修改上传端口

打开 `platformio.ini`，找到：

```
upload_port = COM12
```

改成第 1.3 步看到的端口，并加一行串口监视端口：

```
upload_port = /dev/cu.usbserial-0001
monitor_port = /dev/cu.usbserial-0001
```

【已核对代码】工程原有配置如下：

- 默认环境 `HoloCubic_AIO_Releases`，另有调试环境 `HoloCubic_AIO_Debug`。
- `upload_speed = 921600`，`monitor_speed = 115200`。
- 分区表 `partitions-no-ota.csv`，即**没有 OTA**，只能用 USB 刷机。

  ⚠️ **ESP32-PICO-D4 必须用 DIO + 40 MHz**：本分支的 `platformio.ini` 已改为 `board_build.flash_mode = dio`、`board_build.f_flash = 40000000L`。原仓库是 QIO 80 MHz，实测在 PICO-D4 上**反复重启**【用户实机确认】。

### 3.3 编译 & 上传

VS Code 底部蓝色状态栏上的几个按钮：

- ✓：Build，编译。第一次会下载 ESP32 平台，较慢。
- →：Upload，编译并烧录。
- 🔌：Serial Monitor，查看串口日志。

不想点按钮，也可以用终端。终端里的 `pio` 命令需要在 PlatformIO 侧边栏 → `Miscellaneous → New Terminal` 里打开才可用：

```
pio run -e HoloCubic_AIO_Releases              # 编译
pio run -e HoloCubic_AIO_Releases -t upload    # 上传
pio device monitor -b 115200                   # 串口日志，Ctrl+C 退出
```

本补丁在 Linux 上实测编译通过：Flash 90.8%（3,629,796 / 3,997,696 字节，剩余约 359 KB），RAM 36.4%（静态，119,300 字节）【已编译验证；Mac 上未实机验证】。

### 3.4 上传卡住 / 进不了下载模式

- 日志一直 `Connecting........_____` 或报 `Timed out waiting for packet header`：
  1. 关闭正在运行的串口监视器，它会占用串口。
  2. 把 `upload_speed` 改为 `460800` 或 `115200` 再试。
  3. 如果板子上有 BOOT/IO0 按键：按住它，再点上传，看到 `Writing at…` 后松开。很多 HoloCubic 主板靠自动下载电路，没有按键；这时换线、换口，并检查焊接。
- 官方 Q&A 原话：用 AIO 上位机刷机卡住不动，请检查主板（不要怀疑工具或固件）。

### 3.5 不装 VS Code 也能刷：预编译固件包 + flash.sh

仓库 Release 页提供 `holocube_flash.tgz`，里面有：

- `bootloader_dio_40m.bin`、`partitions.bin`、`boot_app0.bin`、`firmware.bin`
- `flash.sh`：一键刷写，已设置 DIO/40m，写入地址依次为 0x1000、0x8000、0xe000、0x10000
- `SHA256SUMS`

```
tar xzf holocube_flash.tgz && cd holocube_flash
./flash.sh /dev/cu.usbserial-0001      # 不写端口会自动探测
```

`flash.sh` 默认使用 `~/.holocube-pio/bin/esptool`，也可以用 `ESPTOOL=/path/to/esptool ./flash.sh` 指定。命令里用的是 `write-flash` 写法，需要较新的 esptool（4.8 以上 / 5.x）。老版本 esptool 要写成 `write_flash`【未在 Mac 实机验证】。

官方上位机 AIO_Tool 主要面向 Windows，Mac 用户不需要。

### 3.6 串口监视，确认开机正常

打开 Serial Monitor（115200），按一下板子复位或重新插线，正常会看到：

- `Initialization MPU6050 now, Please don't move.`【已核对代码】：此时**把小电视平放、别动**，它在做陀螺仪自动校准。默认开启自动校准（`auto_calibration_mpu = 1`），每次开机执行。
- 如果没插 SD 卡，会有挂载失败的提示，**不影响开机**。README 原文：无论是否插 TF 卡、MPU6050 是否正常、是否连 Wi-Fi，都不影响系统启动和显示。

---

## 4. 手势基础（所有 APP 通用）

【已核对代码 + README】：

| **手势** | **在菜单里** | **在 APP 里** |
|---|---|---|
| 左倾 / 右倾 | 切换到下一个 / 上一个 APP | 由 APP 自己决定 |
| **向前倾并保持约 1 秒** | **进入** APP | — |
| **向后仰并保持约 1 秒** | — | **退出** APP（返回菜单） |

- 进入和退出需要连续 3 次检测到同一方向（每 200 ms 采样一次），所以要**保持住**。
- 本分支把左右切换改为需要**连续 2 次（约 0.4 秒）**同向才算，阈值也调高了，详见 7.3。
- 如果方向是反的，可以在 Web 设置里调整「操作方向（0~15可选）」。
- **开机时请把小电视平放、别碰它**，大约等 2 秒，它在做陀螺仪校准和静止零点采样。

  USB 模式**不需要配置 Wi-Fi**。如果以后想用 Wi-Fi 模式，见附录 8.6。

---

## 5. 进入 AIStatus

1. 菜单**第一个 APP** 就是 **AIStatus**（图标：白色胶囊 + 三个圆点），前倾 1 秒进入。
2. 不进入也可以：电脑一发状态，小电视会**自动切换进 AIStatus**。
3. 没有数据时显示 `no data` / `waiting for USB...`，底部显示 `USB serial 115200`。
4. 屏幕布局（240×240）：

- 顶部：agent 名字。
- 中部：彩色圆点 + 状态大字，下面是消息文本（最多约 3 行）。
- 下方：当前状态持续时间，以及其他 agent 的小列表（最多 4 个）。

1. 退出：后仰 1 秒，RGB 灯恢复原来的灯效。之前收到的状态会保留，下次进入直接显示。
2. 想开机就进入：在 Web 设置的「开机自启的APP名字」里填 `AIStatus`。这需要 WebServer 热点，见 8.6；USB 模式下也可以不设，靠自动切换。

### 5.2 中文显示（已内置，无需 SD 卡）

中文字体已经**直接编译进固件**，不需要 SD 卡，刷完固件就能显示中文和常用 emoji【已核对代码 / 已编译】。

- **字库内容**：ASCII、GB2312 一级常用汉字 3755 个、中文标点、常用符号（← → ✓ ● ★ …）、约 80 个黑白 emoji（✅❌⚠🔥💡🤔🎉🚀👍 等），共 4042 个字形。
- **规格**：16px，**4bpp 抗锯齿 + LVGL 字体压缩**，约 435 KB。用 `lv_font_conv` 从 Noto Sans SC（Medium）+ Noto Emoji 生成，文件是 `src/app/ai_status/ai_cn16.c`。
- **不占内存**：字体数据是 `const`，留在 Flash 里，不会复制到 RAM。已用符号表确认字形数据位于 `.flash.rodata`。
- **哪里用中文字体**：agent 名称、消息文本、小列表。状态大字（thinking/done 等）、计时和 IP 仍用内置英文字体。字库里没有的字符会回退到 Montserrat。
- **为了腾出 Flash 空间，关闭了 LHLXW APP**（它带有大量图片资源，约占 400 KB）。如果需要它，可以在 `src/app/app_conf.h` 里把 `APP_LHLXW_USE` 改回 1，但这样 Flash 会接近满（约 98.7%）。
- **电脑端脚本会预先处理文本**：字库里没有的 emoji 替换成 `[庆祝]`、`[测试]` 这类文字或直接删掉，并按 UTF-8 字符边界截断到 120 字节。固件端也按字符边界截断，不会切出半个汉字。
- **生僻字**：不在一级字库里的字会显示为空白。
- **重新生成字体**：见 `AIO_Firmware_PIO/tools/build_ai_font.sh`。

---

## 6. 电脑端接入

以下命令都在「终端」中执行。`ai_status_bridge` 换成你实际解压的位置。

### 6.1 安装通知脚本

```
mkdir -p ~/.holocube
cp ai_status_bridge/holocube_notify.sh ai_status_bridge/holocube_serial_send.py \
   ai_status_bridge/sanitize_text.py ai_status_bridge/ai_cn16_charset.txt \
   ai_status_bridge/test_states.sh ai_status_bridge/grok_wrapper.sh ~/.holocube/
chmod +x ~/.holocube/*.sh ~/.holocube/*.py
```

- **串口自动探测**：顺序是 `/dev/cu.usbserial-*`、`cu.wchusbserial*`、`cu.SLAB_USBtoUART*`。
- **手动指定端口**（Mac 上插了多个串口设备时建议这样做）： `bash ls /dev/cu.* echo /dev/cu.usbserial-0001 > ~/.holocube_port # 或临时: export CUBE_PORT=/dev/cu.usbserial-0001`
- **需要带 pyserial 的 Python**：脚本依次尝试 `~/.holocube-pio/bin/python3`、`~/.platformio/penv/bin/python3`、`python3`。PlatformIO 自带的 Python 一般已包含 pyserial。

脚本特点【已在 Linux 用虚拟串口测试】：

- 在后台发送，hook 本身约 0.05 秒就返回。

- 多个 hook 同时触发时排队，排队超过 1.5 秒的消息会被丢弃。
- 端口被占用或拔掉线时静默放弃，错误写到 `$TMPDIR/holocube_notify.err`。
- 打开串口时 DTR/RTS 保持释放，目的是**不让 ESP32 被复位**【未在 Mac 实机验证】。

### 6.2 连通测试

**先关闭串口监视器**（它会占用端口），再运行：

```
~/.holocube-pio/bin/python3 ~/.holocube/holocube_serial_send.py --check
```

应打印 `/dev/cu.usbserial-xxxx AIS:PONG`。然后发一个状态：

```
~/.holocube/holocube_notify.sh thinking claude "你好，方块 🚀"
```

小电视应切到 AIStatus，显示 `claude / thinking / 你好，方块 🚀`，灯变蓝。

### 6.3 跑一遍所有状态

```
~/.holocube/test_states.sh
```

### 6.4 Claude Code

事件名已对照 Claude Code 官方 hooks 文档（code.claude.com/docs/en/hooks）核对。

1. 打开（不存在就新建）`~/.claude/settings.json`： `bash mkdir -p ~/.claude && open -a TextEdit ~/.claude/settings.json` TextEdit 要用「格式 → 制作纯文本」。用 VS Code 打开更方便：`code ~/.claude/settings.json`。
2. **文件是空的**：把 `ai_status_bridge/claude_settings_snippet.json` 的全部内容粘进去。 **文件已有内容**：只把其中 `"hooks": {...}` 这一段合并到最外层的大括号里，注意逗号。已经有 `hooks` 的，把各事件数组合并。
3. 映射关系：

| Claude 事件 | 何时触发 | 小电视状态 |
|---|---|---|
| `UserPromptSubmit` | 你发出提问 | thinking |
| `PreToolUse` | 调用工具前（读写文件、跑命令） | working |
| `Notification` | 需要你确认权限 / 闲置等待输入 | waiting |
| `Stop` | 回答结束 | done |
| `StopFailure` | 因 API 错误结束 | error |

1. 每个 hook 都设置了 `"async": true`（后台运行，不拖慢 Claude）。
2. 在 Claude Code 中输入 `/hooks`，能看到这些 hook 就说明加载成功。

### 6.5 Codex CLI

对照 OpenAI Codex 官方配置文档核对：

- `notify` 只能写在**用户级** `~/.codex/config.toml`，写在项目里的 `.codex/config.toml` 会被忽略。
- 目前只有 `agent-turn-complete`（一轮结束）这一个事件，JSON 作为最后一个参数传给程序。

```
whoami          # 记下你的用户名
mkdir -p ~/.codex && code ~/.codex/config.toml
```

在文件**最上方**（任何 `[xxx]` 小节之前）加一行，把 `你的用户名` 替换掉：

```
notify = ["/Users/你的用户名/.holocube/holocube_notify.sh", "--codex"]
```

- 效果：Codex 每答完一轮，小电视显示 `codex / done`，消息为最后一句回复。
- 局限：`notify` **没有**「等待确认」事件，所以 Codex 不会显示 waiting/working。Codex 新版本另有生命周期 hooks 机制，本指南未配置【未验证】。

### 6.6 Grok

根据 docs.x.ai 文档【未实机验证】：

- Grok CLI（`grok`）支持 hooks，个人配置放在 `~/.grok/hooks/*.json`，格式与 Claude 相同。
- 它**默认也读取 `~/.claude/settings.json` 里的 Claude hooks**，所以完成 6.4 后，Grok 很可能已经自动生效。
- 脚本检测到环境变量 `GROK_HOOK_EVENT` 时，会自动把名字显示为 `grok`。
- 检查是否加载：在 Grok 里输入 `/hooks`，或在终端运行 `grok inspect`。
- 如果想单独配置，把 `claude_settings_snippet.json` 复制为 `~/.grok/hooks/holocube.json`。注意不要和 Claude 那份同时生效，否则会重复上报。
- 不用 hooks 的办法：用包装脚本启动，开始显示 working，退出显示 done 或 error： `bash ~/.holocube/grok_wrapper.sh grok` 这个办法适用于任何命令行工具。

---

## 7. 问题排查

### 7.1 方块端（硬件 / 固件）

| 现象 | 可能原因 | 解决 |
| --- | --- | --- |
| `ls /dev/cu.*` 没有新设备 | 充电线不能传数据；缺驱动；USB 扩展坞问题 | 换数据线；直插 Mac；按 1.3 装 CP210x/CH340 驱动并在「隐私与安全性」允许，然后重启 |
| 上传报 `Timed out waiting for packet` | | |

| 现象 | 可能原因 | 解决 |
| --- | --- | --- |
| `header` / 卡在 `Connecting...` | 串口被监视器占用；自动下载电路或焊接问题；速率太高 | 关监视器；`upload_speed` 降到 115200；有 BOOT 键就按住再上传；检查主板焊接（官方 Q&A：刷机卡住先查主板） |
| `Resource busy` / `could not open port` | 串口被其他程序占用 | 关闭 Serial Monitor、其他串口工具、Arduino IDE |
| 反复重启，尤其在连 Wi-Fi 时；发热严重 | 供电不足；屏幕板/主板 LDO 方案发热、压降大 | 换供电充足的 USB 口或充电头；参考仓库 `硬件版本说明.md`：Metal 屏幕板 v1.2+ 加了一路 LDO 分担电流，v2.0/v2.1 可改焊 DC-DC 方案（此时去掉主板 LDO）以根治发热 |
| 白屏、花屏、黑屏但串口正常 | 屏幕焊接/排线问题；屏幕型号不符（需 1.3 寸 ST7789 240×240）；`TFT_eSPI` 配置被改 | 检查屏幕焊接；不要随意修改 `lib/TFT_eSPI/User_Setup.h`；背光引脚在 `src/common.h` 中为 IO5（PEAK 版为 IO12） |
| 串口出现 `sdcard_mount(): f_mount failed: (3) The physical drive cannot work` | 没插卡、卡不兼容、卡槽焊接问题、格式不对 | AIStatus 不需要 SD 卡，**可忽略**；其他 APP 需要时：换官方推荐的 32G 卡并格式化为 FAT32，检查卡槽焊接（SD 用 SPI：SCK14/MISO26/MOSI13/SS15）。代码里 SD 的 SPI 时钟设为 80 MHz，个别卡不稳定属已知兼容性问题（官方 issue 有此报错） |
| （Wi-Fi 模式）一直显示 `WiFi...` | 没配 Wi-Fi；5G；需要网页认证的网络；密码错 | 按 8.6 配网；或直接用默认的 USB 模式 |
| 刷完反复重启（串口不停打印 boot 信息 / `flash read err`） | 用了 QIO 80 MHz | 确认 `platformio.ini` 为 `dio` + `40000000L`，或用 `flash.sh` 刷写 |
| `--check` 没有回复 / `cannot open` | 串口被监视器占用；端口不对；固件不是新版本 | 关闭 Serial Monitor；`ls /dev/cu.*` 确认端口并写入 `~/.holocube_port`；重新刷 USB 版固件 |
| 每发一次状态小电视就重启 | 打开串口时触发了自动复位电路 | 反馈给我们；临时办法：`stty -f /dev/cu.xxx -hupcl` 后再试【未验证】 |
| 菜单自己左右乱切、日志不停出现 SHAKE | 开机校准时被移动；静止零点偏大；阈值过低 | 本分支已调高阈值并加了静止零点补偿（见 7.3）；开机保持平放不动；仍不行就打开 `IMU_DEBUG_RAW` 看数值 |
| 手势不灵、乱切换、进不去 APP | 开机自动校准时被移动；MPU6050 焊接/I2C（SDA32/SCL33）问题；方向设置不对 | 开机时平放不动等校准完成；倾斜要**保持约 1 秒**；Web 设置里调整「操作方向（0~15可选）」；仍不行检查 MPU6050 焊接（官方 Q&A：IIC 焊接有问题会导致切换不了页面） |
| 中文全部显示为方块/空白 | 刷的不是带内置中文字体的新固件 | 确认用 `ai-status` 分支最新代码重新编译上传 |
| 个别汉字显示空白 | | |

| 现象 | 可能原因 | 解决 |
| --- | --- | --- |
| | 该字不在 GB2312 一级字库（3755 字）里 | 属正常限制；需要的话扩充 `tools/sc.txt` 重新生成字体 |
| emoji 变成 `[庆祝]` 之类的文字或消失 | 字库只含约 80 个常用 emoji，其余由电脑端脚本替换或删除 | 属设计行为；可在 `sanitize_text.py` 里修改映射 |
| 菜单里找不到 LH&LXW APP | 为了给中文字体腾出 Flash 空间，已在 `app_conf.h` 关闭 | 需要时把 `APP_LHLXW_USE` 改为 1（Flash 会接近满） |
| 编译报 Flash 空间不足 | 又打开了被关闭的 APP，或加了新功能 | 在 `app_conf.h` 关闭不用的 APP |
| 编译报 `arduino.h: No such file` / `EMOJI.h` | 原仓库头文件大小写不一致，区分大小写的文件系统上会报错 | 本补丁已修正；macOS 默认不区分大小写，一般不会遇到 |
| 编译报 Flash 空间不足（`region flash overflowed`） | 开了太多 APP | 在 `src/app/app_conf.h` 中把不用的 APP 宏改成 0 |
| 第一次编译特别慢或下载失败 | 首次需要下载 espressif32 平台和工具链 | 保持网络通畅，重试；必要时开代理 |
| 刷新固件后 Wi-Fi 设置丢了 | 分区布局变化导致 Flash 文件系统被重建 | 按第 4 章重新配网 |

## 7.2 电脑端

| 现象 | 原因 | 解决 |
| --- | --- | --- |
| `--check` 正常，AI 工具触发后屏幕没变化 | 脚本没有执行权限；hooks 没加载；串口被监视器占用 | `chmod +x ~/.holocube/*.sh ~/.holocube/*.py`；Claude 里输入 `/hooks` 检查；查看 `$TMPDIR/holocube_notify.err` |
| 想单独测试脚本 | — | `~/.holocube/holocube_notify.sh working claude "test"` |
| `settings.json` 改完 Claude 报配置错误 | JSON 格式错（逗号、括号） | 把内容粘到在线 JSON 校验器检查，或运行 `python3 -m json.tool ~/.claude/settings.json` |
| Codex 没反应 | `notify` 写在了某个 `[小节]` 下面；路径里用了 `~`；写在了项目配置里 | 放到文件最顶部；用 `/Users/用户名/...` 绝对路径；写在 `~/.codex/config.toml` |
| 同一事件显示两次 / 名字变来变去 | Claude 与 Grok 共用一份 hooks，或 Grok 同时加载了两份 | 只保留一份配置 |

### 7.3 手势防误触改动说明（本分支）

现象：小电视静止放在桌上时，日志不停出现 SHAKE，还会偶尔出现 TURN_LEFT/TURN_RIGHT，菜单自己切换。

改动【已编译，未实机验证效果】，位于 src/driver/imu.h / imu.cpp：

1. **修复原代码 bug**：前后方向的「震动」判断误用了上一次的结构体 action_info.v_ax（从不更新），改为本次采样值。
2. **静止零点补偿**：开机校准后采 20 个样，把静止时 ax/ay 的残余值记为零点并减掉。残余超过 3000 时不补偿，避免开机时拿在手上导致零点错误。串口会打印 [IMU] rest ax=.. ay=.. -> bias ...。
3. **阈值调高**（±2g 量程下 16384 ≈ 1g）：

| 判断 | 原阈值 | 新阈值 |
|---|---|---|
| 左右倾 | 4000（约 14°） | 5000（约 18°） |
| 前倾/后仰 | 5000（约 18°） | 5500（约 20°） |
| 震动 SHAKE | 1000（约 3.5°） | 2500（约 9°） |

1. **左右切换需确认**：需要连续 2 次采样（约 0.4 秒）同向才切换。IMU_TURN_CONFIRM 1 可恢复原行为。

还要注意：

- 调试时可把 IMU_DEBUG_RAW 设为 1，每秒打印一次数值，便于把日志发来分析。
- 日志里的 CPU 80 MHz 是「节能模式」（power_mode=0）的设置，与手势误触基本无关。手势每 200 ms 由定时器触发采样一次，与 CPU 频率无关。

---

## 8. 附录

### 8.1 状态对照表

| state | 含义 | 屏幕/灯颜色 |
|---|---|---|
| idle | 空闲 | 灰 #808080 |
| thinking | 思考中 | 蓝 #3D7BFF |
| working | 执行工具/写代码 | 橙 #FFB000 |
| waiting | 等你确认/输入 | 粉 #FF40C0 |
| done | 完成 | 绿 #20D060 |
| error | 出错 | 红 #FF3030 |

## 8.2 协议说明（USB 串口，115200 8N1）

每条消息是**一行 JSON + 换行（`\n`）**，UTF-8 编码：

```
{"agent": "claude", "state": "working", "text": "正在修改 main.cpp"}
```

| 字段 | 说明 |
|---|---|
| `agent` | 必填。名字，最长 23 字节（UTF-8 安全截断），不区分大小写匹配 |
| `state` | 必填。必须是上表 6 个值之一（大小写不敏感） |
| `text` | 可选。超过 127 字节会按字符边界截断并加 `...` |

回复（也是一行，可能夹在日志中间，以 `AIS:` 开头）：

- `AIS:OK`：已更新。
- `AIS:ERR bad json` / `AIS:ERR bad state` / `AIS:ERR need agent+state`：内容有误。
- 发送 `{"ping":1}` 回复 `AIS:PONG`。
- 亮度（v0.2.1 起）：发送 `{"brightness":N}`（N 为 0–100 的整数）立即调整背光，并写入 `/sys.cfg`（与 Web 设置页同一保存路径），重启后保持；回复 `AIS:OK brightness=N`，超范围回复 `AIS:ERR brightness must be 0-100`。发送 `{"get":"brightness"}` 回复 `AIS:brightness=N`。这两条命令**不会**自动切到 AIStatus。注意 0 = 背光全关（屏幕全黑），一般用 10 以上。

固件只处理**以 `{` 开头并且包含 `"agent"`、`"ping"` 或 `"brightness"` 的行**，其他任何输入都忽略。单行最长 383 字节，超长行整行丢弃。

- 同时最多记录 4 个 agent，满了替换最久没更新的那个。
- 最近一次更新的 agent 显示为主画面。

可选 Wi-Fi 模式（编译时 `AI_STATUS_USE_WIFI 1`）：`POST http://<IP>:8080/status` 发同样的 JSON，`GET /status` 查询。

## 8.3 常用命令速查

```
ls /dev/cu.*                                                    # 找串口
pio run -e HoloCubic_AIO_Releases -t upload                     # 编译+上传
pio device monitor -b 115200                                    # 串口日志（用完记得关，否则状态发不过去）
~/.holocube-pio/bin/python3 ~/.holocube/holocube_serial_send.py --check   # 连通测试
~/.holocube-pio/bin/python3 ~/.holocube/holocube_serial_send.py --brightness 50   # 亮度设为 50 并保存
~/.holocube-pio/bin/python3 ~/.holocube/holocube_serial_send.py --brightness      # 查询当前亮度
~/.holocube/holocube_notify.sh done claude "hi"                 # 手动发送
~/.holocube/test_states.sh                                      # 全状态测试
./flash.sh /dev/cu.usbserial-0001                               # 用预编译包刷机
```

## 8.4 相关代码位置

| 内容 | 路径 |
| --- | --- |
| AIStatus APP | `AIO_Firmware_PIO/src/app/ai_status/` |
| APP 开关 | `AIO_Firmware_PIO/src/app/app_conf.h` |
| APP 安装顺序 | `AIO_Firmware_PIO/src/HoloCubic_AIO.cpp` |
| 热点名 / AP IP | `AIO_Firmware_PIO/src/network.cpp` |
| 串口状态解析 / WiFi 开关宏 | `src/app/ai_status/ai_status.cpp`、`ai_status.h`（`AI_STATUS_USE_WIFI`、`AI_STATUS_AUTO_ENTER`） |
| 手势阈值 | `src/driver/imu.h`（`IMU_*` 宏） |
| 网页配置 | `AIO_Firmware_PIO/src/app/server/` |
| 引脚定义 | `AIO_Firmware_PIO/src/common.h` |
| 手势识别 | `AIO_Firmware_PIO/src/driver/imu.cpp` |

## 8.5 尚未验证的事项（如实列出）

1. USB 版 AIStatus（串口解析、自动切换进 APP）**未在真机运行**。已确认的只有：编译通过，以及用户实测 DIO/40 MHz 能开机。
2. 在 macOS 上打开串口会不会复位 ESP32，取决于 USB 转串口芯片和自动复位电路，需要实测。
3. 手势阈值和零点补偿的实际效果需要实测，可能还要再调。
4. `flash.sh` 用的 `write-flash` 写法依赖较新的 esptool。
5. Grok 读取 Claude hooks 只根据官方文档，未实测。

## 8.6 （可选）Wi-Fi 模式与 WebServer 配网

**配网**【已核对代码】：

- WebServer APP 会放出热点 `HoloCubic_AIO`（无密码），地址固定为 `http://192.168.4.2`。
- 设置页的 `WiFi SSID_0(2.4G)` / `WiFi Passwd_0` 保存在 Flash。
- 同一页可以设置「开机自启的APP名字」。

**改用 Wi-Fi 模式**：

1. 在 `src/app/ai_status/ai_status.h` 中设置 `#define AI_STATUS_USE_WIFI 1`，重新编译。
2. 电脑上设置 `export CUBE_IP=<屏幕底部显示的IP>`，脚本就改走 HTTP。

注意：需要网页认证（captive portal）的 Wi-Fi 无法使用，必须是普通 WPA/WPA2 的 2.4G 网络。